

Hands-on Lab: Populating a Data Warehouse using PostgreSQL



Estimated time needed: 1 hour

Purpose of the Lab:

The lab is designed to provide hands-on experience in creating and managing a production database using PostgreSQL within the IBM Skills Network Labs (SN Labs) Cloud IDE. You will learn how to launch a PostgreSQL server instance, utilize the pgAdmin graphical user interface (GUI) for database operations, and execute essential tasks like creating a database, designing tables, and loading data. The lab focuses on building a foundation in database management by guiding learners through the process of setting up a 'Production' database and populating it with data following a star schema design.

Benefits of Learning the Lab:

Engaging in this lab offers significant benefits for learners seeking to deepen their understanding of database management systems, particularly PostgreSQL. By working through the lab, you will gain practical skills in SQL, database creation, table design, and data manipulation, which are crucial for roles in data engineering, database administration, and data science. The hands-on approach helps in consolidating knowledge of database schemas and SQL queries, thereby enhancing the learner's ability to manage and analyze data effectively in real-world scenarios. Additionally, familiarity with tools like pgAdmin and the Cloud IDE environment adds valuable experience to your skill set, preparing you for advanced database projects and tasks.

Software Used in this Lab

To complete this lab you will utilize the [PostgreSQL Database](#) relational database service available as part of IBM Skills Network Labs (SN Labs) Cloud IDE. SN Labs is a virtual lab environment used in this course.



Database Used in this Lab

Production database is used in this lab.

The production database contains:

- DimCustomer
- DimMonth
- FactBilling

Objectives

In this lab you will:

- Create production related database and tables in a PostgreSQL instance.
- Populate the production data warehouse byloading the tables from Scripts.

Lab Structure

In this lab, you will complete several tasks in which you will learn how to create tables and load data in the PostgreSQL database service using the pgAdmin graphical user interface (GUI) tool.

Data Used in this Lab

The following are the SQL data files used in this lab.

The production database contains:

- [DimCustomer](#)
- [DimMonth](#)
- [FactBilling](#)
- [Star Schema](#)

Task A: Create a database

First, to create a database on a PostgreSQL server instance, you'll first want to actually launch a PostgreSQL server instance on Cloud IDE and open up the pgAdmin Graphical User Interface.

1. Click on the Skills Network extension button on the left side of the window.
2. Open the **DATABASES** drop down menu.
3. Click on **PostgreSQL**
4. Click on the **Create** button. PostgreSQL may take a few moments to start.

The screenshot shows the DBeaver application window. The top menu bar includes 'File', 'Edit', 'Selection', 'View', 'Go', and 'Run'. Below the menu bar is a toolbar with navigation icons. The left sidebar contains a vertical list of icons representing different database types. The main panel is divided into two sections: a list of database connections on the left and a detailed view of the selected 'PostgreSQL' connection on the right. Red annotations highlight three specific elements: a red box with the number '1' around the 'Databases' icon in the sidebar; a red box with the number '2' around the 'Databases' menu item in the top menu bar; and a red box with the number '3' around the 'Create' button in the PostgreSQL connection details panel.

File Edit Selection View Go Run

← → | [Icon]

SKILLS NE... [Icon] [Icon]

✓ DATABASES 2

MySQL INACTIVE

PostgreSQL INACTIVE 3

Cassandra INACTIVE

MongoDB INACTIVE

> BIG DATA

> CLOUD

> EMBEDDABLE AI

> OTHER

Launch Applica...

PostgreSQL x

PostgreSQL

v13.2 | v5

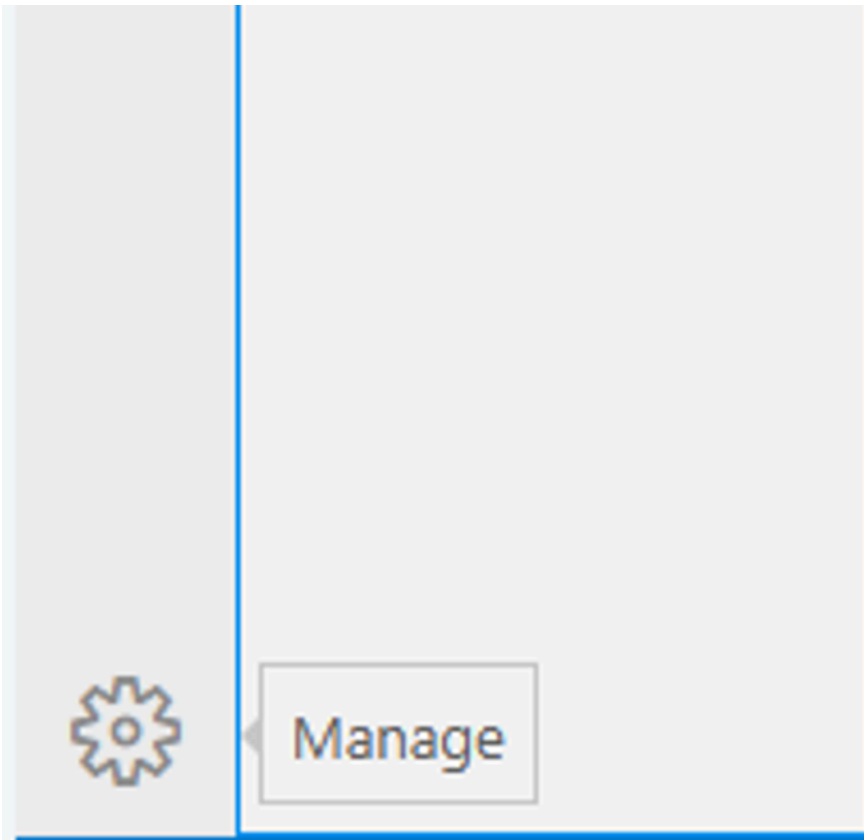
Connect to PostgreSQL environment.

Create

Summary Con

Get started with Pos the Start button.

1



5. Next, open the pgAdmin Graphical User Interface by clicking the **pgAdmin launch** button in the Cloud IDE interface.

FileEditSelectionViewGoRun

PostgreSQL X

PostgreSQL

ACTIVE

v13.2

v5.0

v14.5

Connect to PostgreSQL and pgAdmin dire

Create

De

Summary

Connection Information

Your database and pgAdmin server are no

For more details on how to navigate Postg

You can manage PostgreSQL via:

pgAdmin

Or to interact with the database in the ter

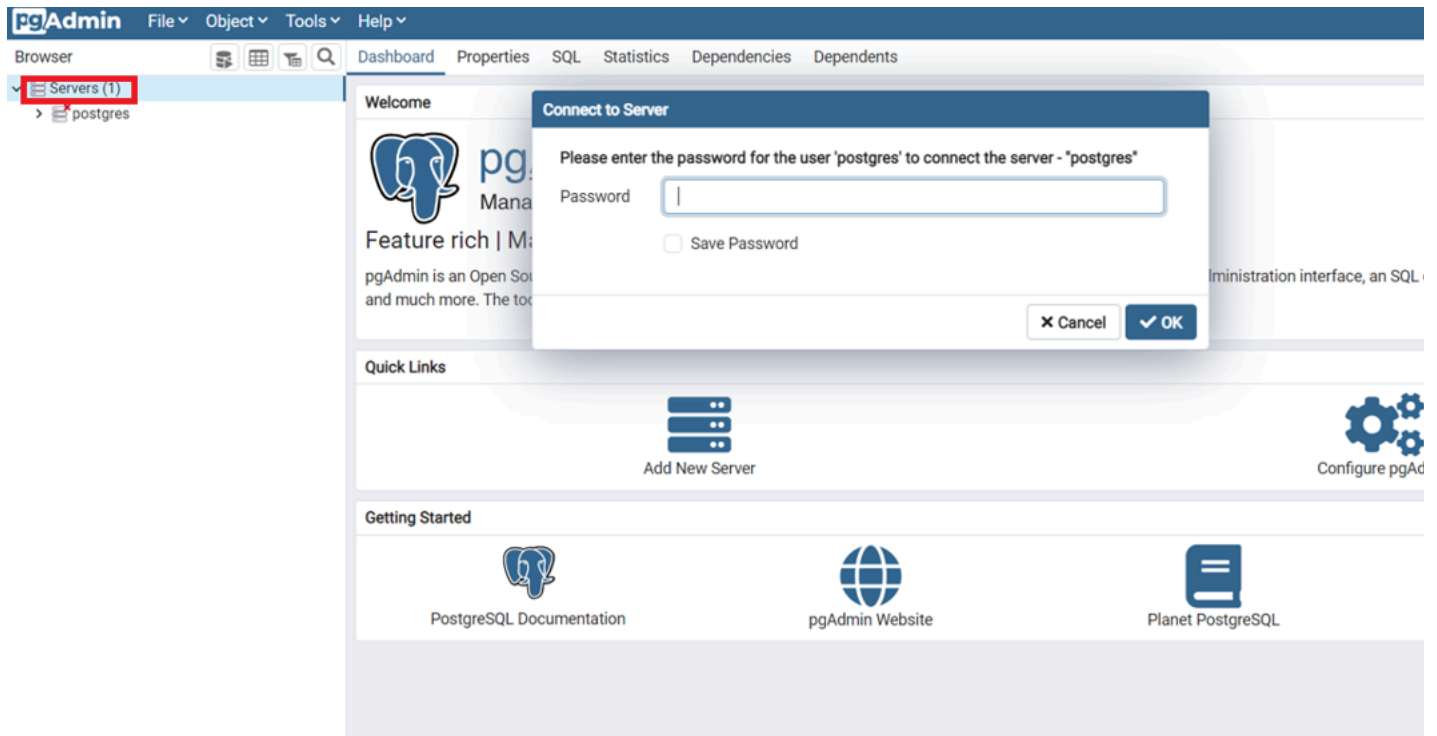
PostgreSQL CLI

New Termin



Manage

6. Once the pgAdmin GUI opens, click on the **Servers** tab on the left side of the page. You will be prompted to enter a password.



7. To retrieve your password, click on the **PostgreSQL** and go to **Conection Information** tab on the top of the interface.

FileEditSelectionViewGoRun

PostgreSQL

X

PostgreSQL

ACTIVE

v13.2

v5.0

v14.5

Connect to PostgreSQL and pgAdmin directly

Create

De

Summary

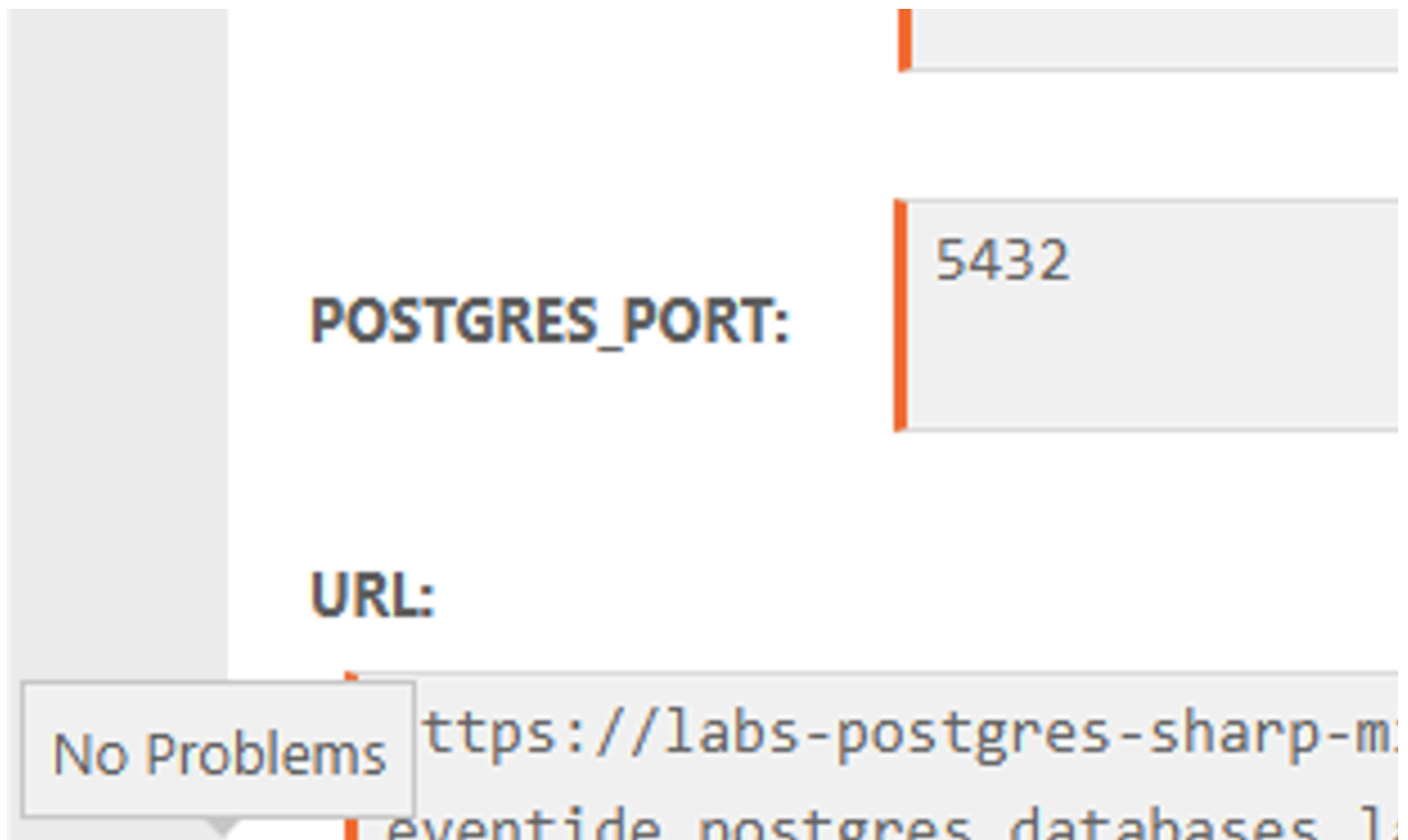
Connection Information

POSTGRES_USERNAME:

postgres

POSTGRES_HOST:

172.21.68.118



8. Scroll down to the session password and click on the **Copy** icon to the right of your password to copy onto your clipboard.

File Edit Selection View Go Run

← → |

 PostgreSQL X

eventide.postgres.databases.la



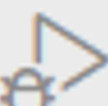
PostgreSQL CLI Command:

export PGPASSWORD=U3hEw8LpGIxE
-p 5432 -U postgres



POSTGRES_COMMAND:

export PGPASSWORD=U3hEw8LpGIxE
-p 5432 -U postgres









POSTGRES_PASSWORD:

POSTGRES_TITLE:

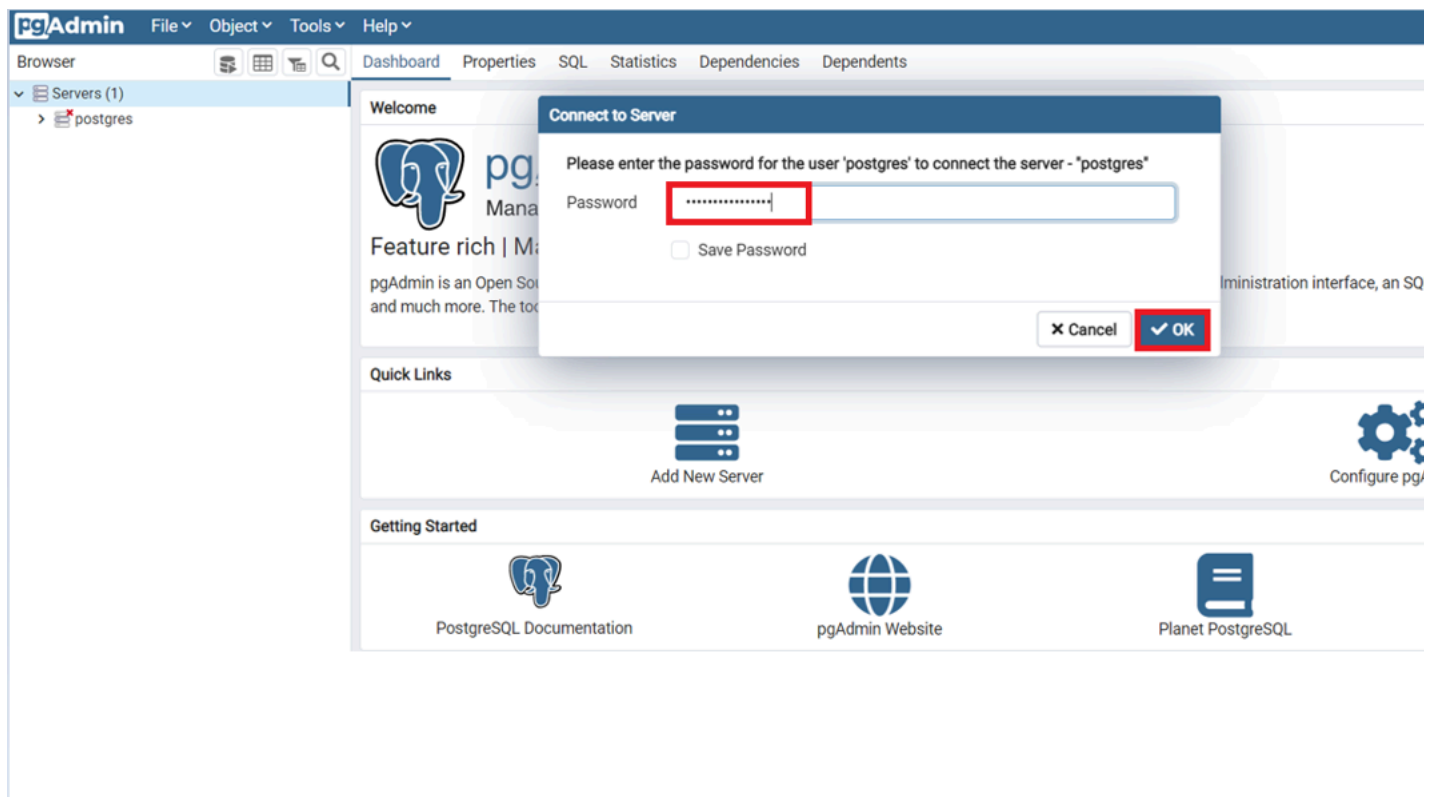
Postgres Datab

POSTGRES_ID:

labs-postgres-sha

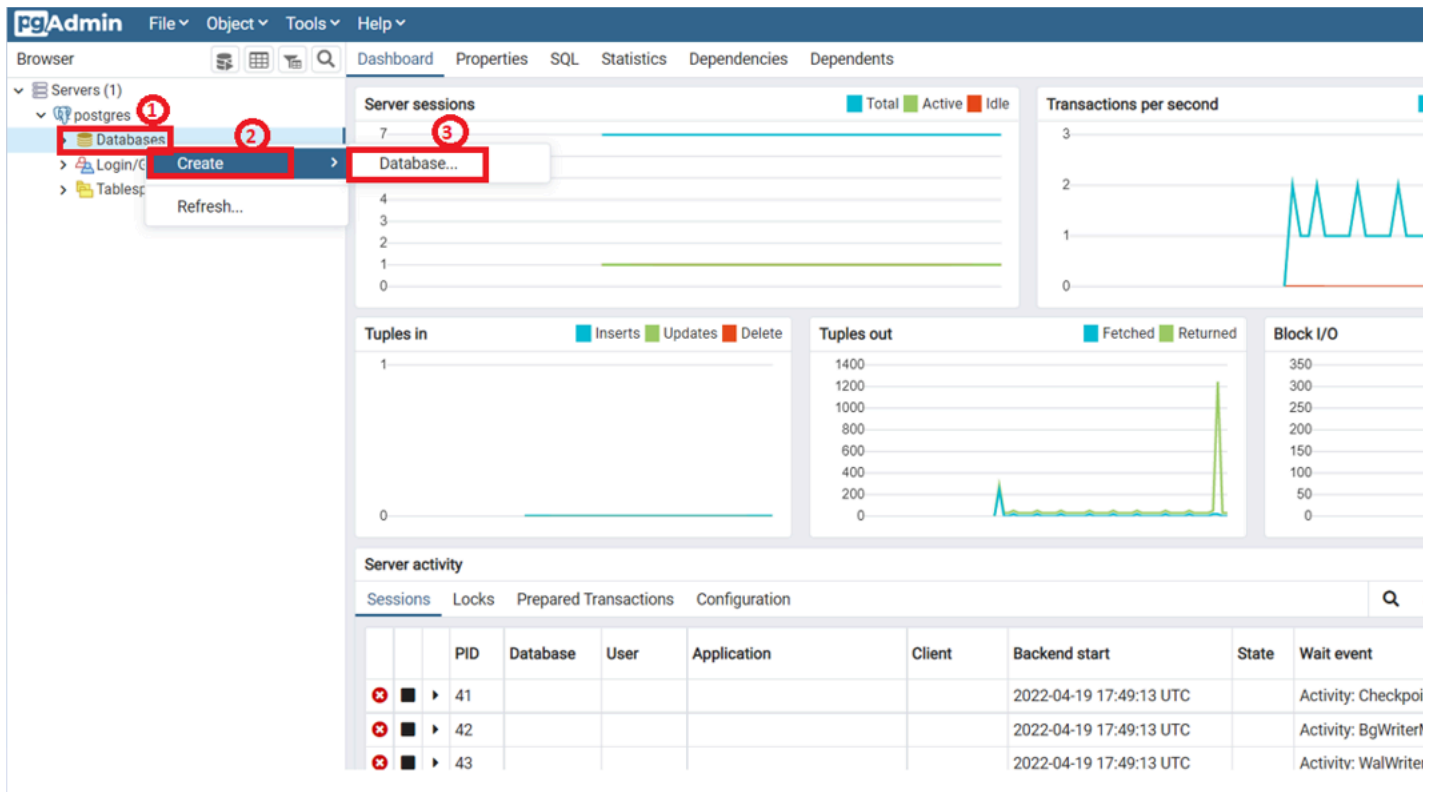


9. Navigate back to the **pgAdmin** tab and paste in your password, then click OK.

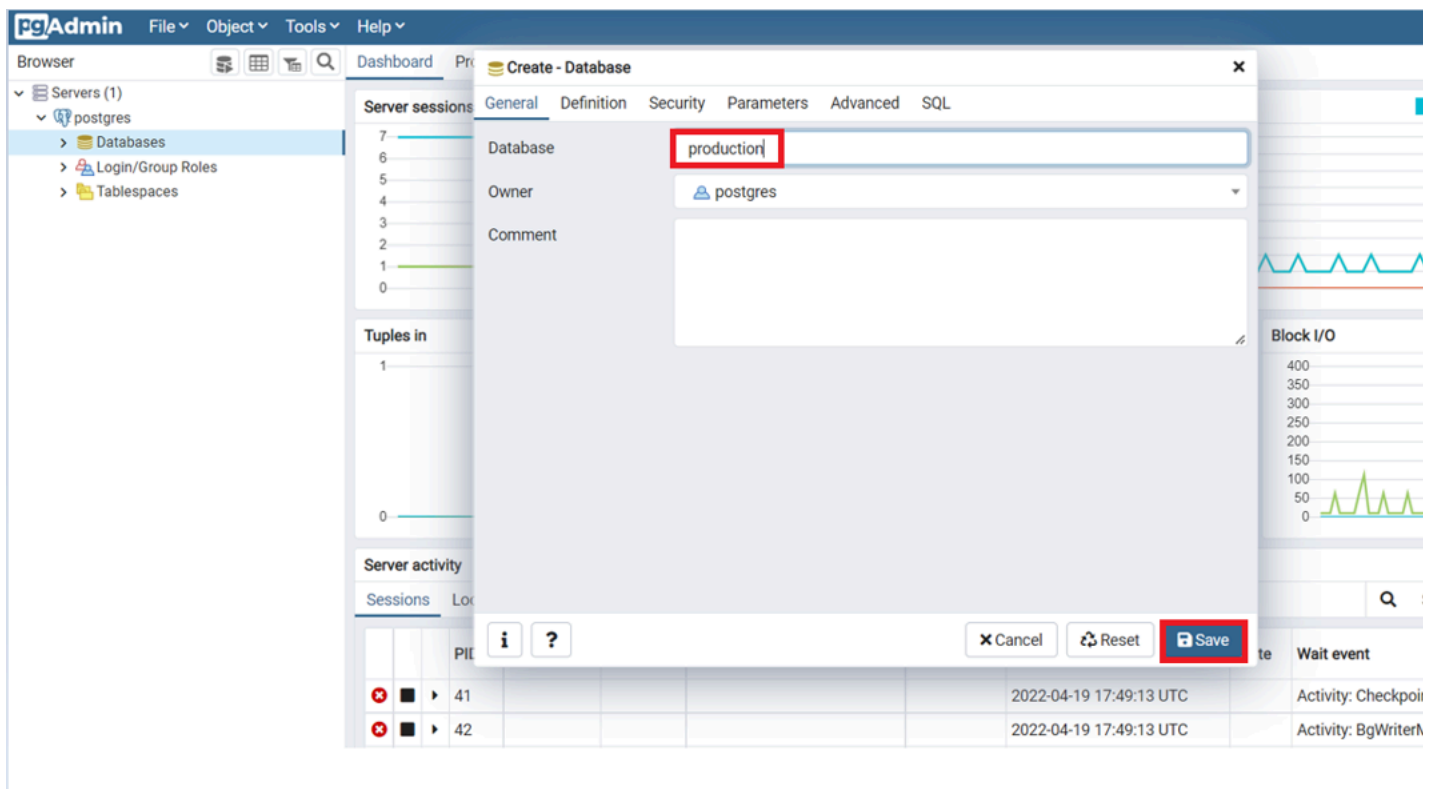


10. You will then be able to access the pgAdmin GUI tool.

11. In the left tree-view, right-click on **Databases> Create > Database**.



In the Database box, type **Production** as the name for your new database, and then click **Save**. Proceed to Task B.



Task B: Create tables

Now, that you have your PostgreSQL service active and have created the **Production database** using pgAdmin, let's go ahead and create a few tables to populate the database and store the data that we wish to eventually upload into it.

1. Click on the Production database and in the top of the page go to **Query tool** and then click on **Open File**. Next a new page pops up called **Select File**. Click on the three dots and choose **Upload** option as shown in the screenshot.

Note: Ensure that you upload the files to this path: `/var/lib/pgadmin/`

pgAdmin File ▾ Object ▾ Tools ▾

Object Explorer     

Dashboard

▾ Servers (1)

▾ PostgreSQL

▾ Databases (2)

▾ **Production**

- >  Casts
- >  Catalogs
- >  Event Triggers
- >  Extensions
- >  Foreign Data Wrappers
- >  Languages
- >  Publications
- >  Schemas
- >  Subscriptions
- >  postgres
- >  Login/Group Roles
- >  Tablespaces

Query  

1

Data Outp



2. In the new blank page that appears drag and drop the **star-schema.sql** file inside the blank page. Once the **star-schema.sql** file is successfully loaded, click on the **X** icon on the right hand side of the page as shown in the screenshot.

pgAdmin File ▾ Object ▾ Tools ▾

Object Explorer     

▾  Servers (1)

▾  PostgreSQL

▾  Databases (2)

▾  Production

- >  Casts
- >  Catalogs
- >  Event Triggers
- >  Extensions
- >  Foreign Data Wrappers
- >  Languages
- >  Publications
- >  Schemas
- >  Subscriptions
- >  postgres
- >  Login/Group Roles
- >  Tablespaces

Dashbo

Query

1

Data Ou



Total re

3. Once you click on the **X** icon a new page appears with the file **star-schema.sql**. Select the **star-schema.sql** file from the list and click the **Select** tab.

pgAdmin File ▾ Object ▾ Tools ▾

Object Explorer     

Dashboard

- ▾  Servers (1)
 - ▾  PostgreSQL
 - ▾  Databases (2)
 - ▾  Production**
 - >  Casts
 - >  Catalogs
 - >  Event Triggers
 - >  Extensions
 - >  Foreign Data Wrappers
 - >  Languages
 - >  Publications
 - >  Schemas
 - >  Subscriptions
 - >  postgres
 - >  Login/Group Roles
 - >  Tablespaces

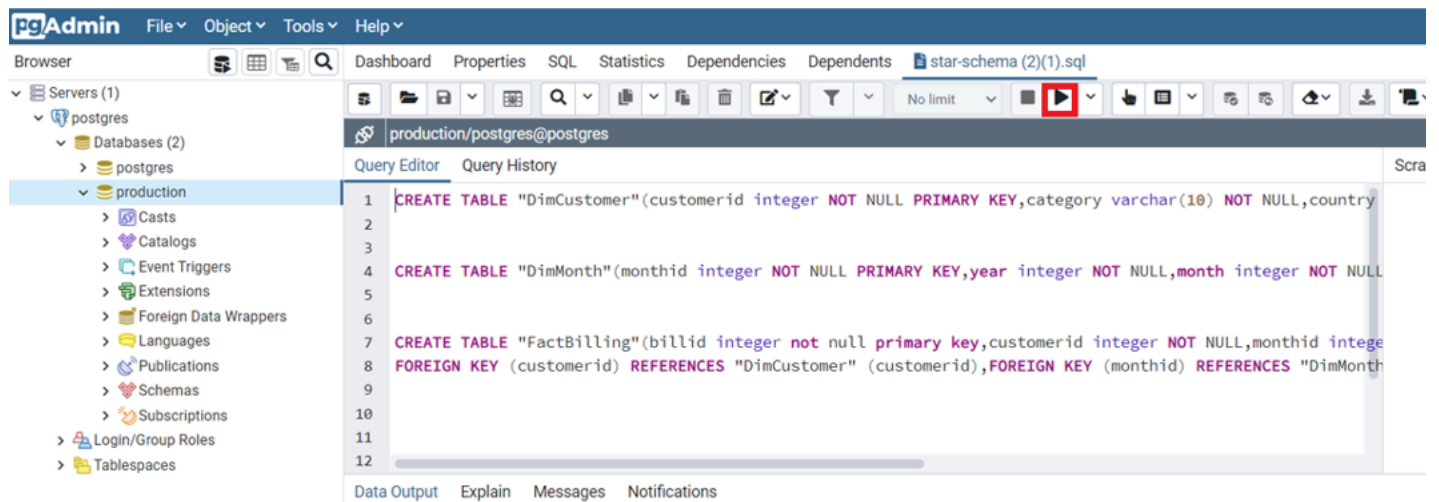
Query

1

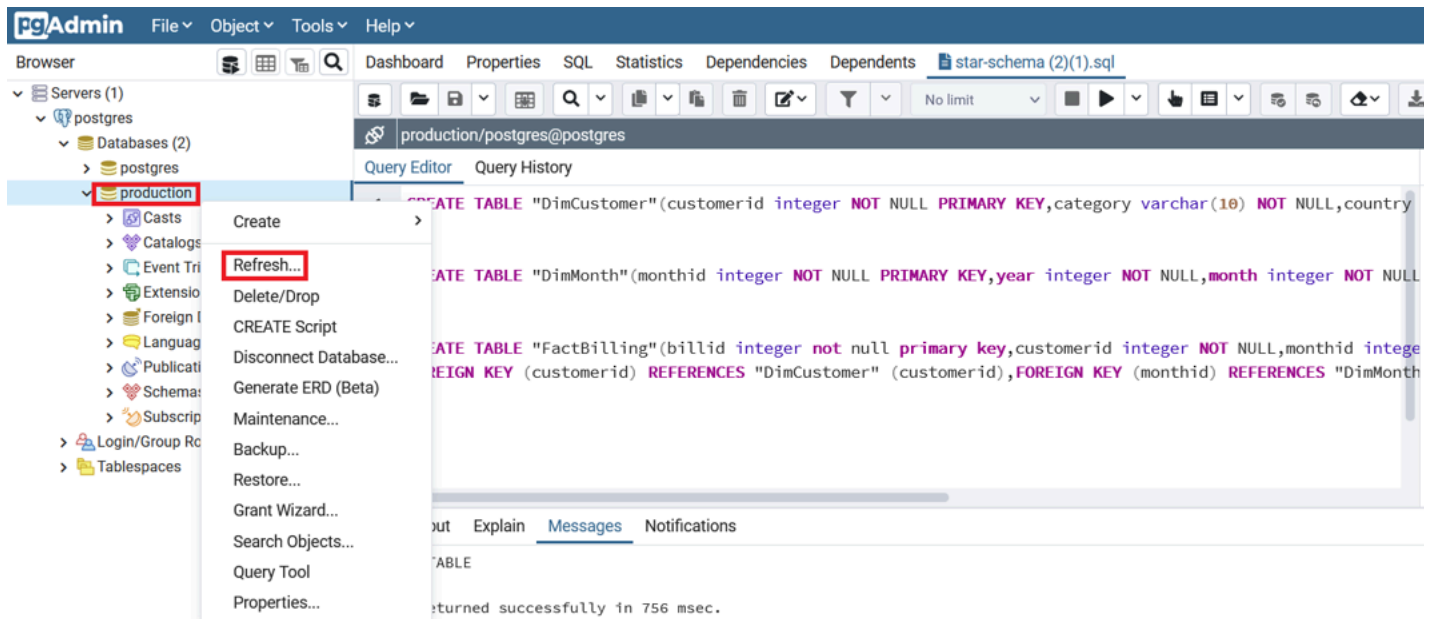
Data Output

Total row

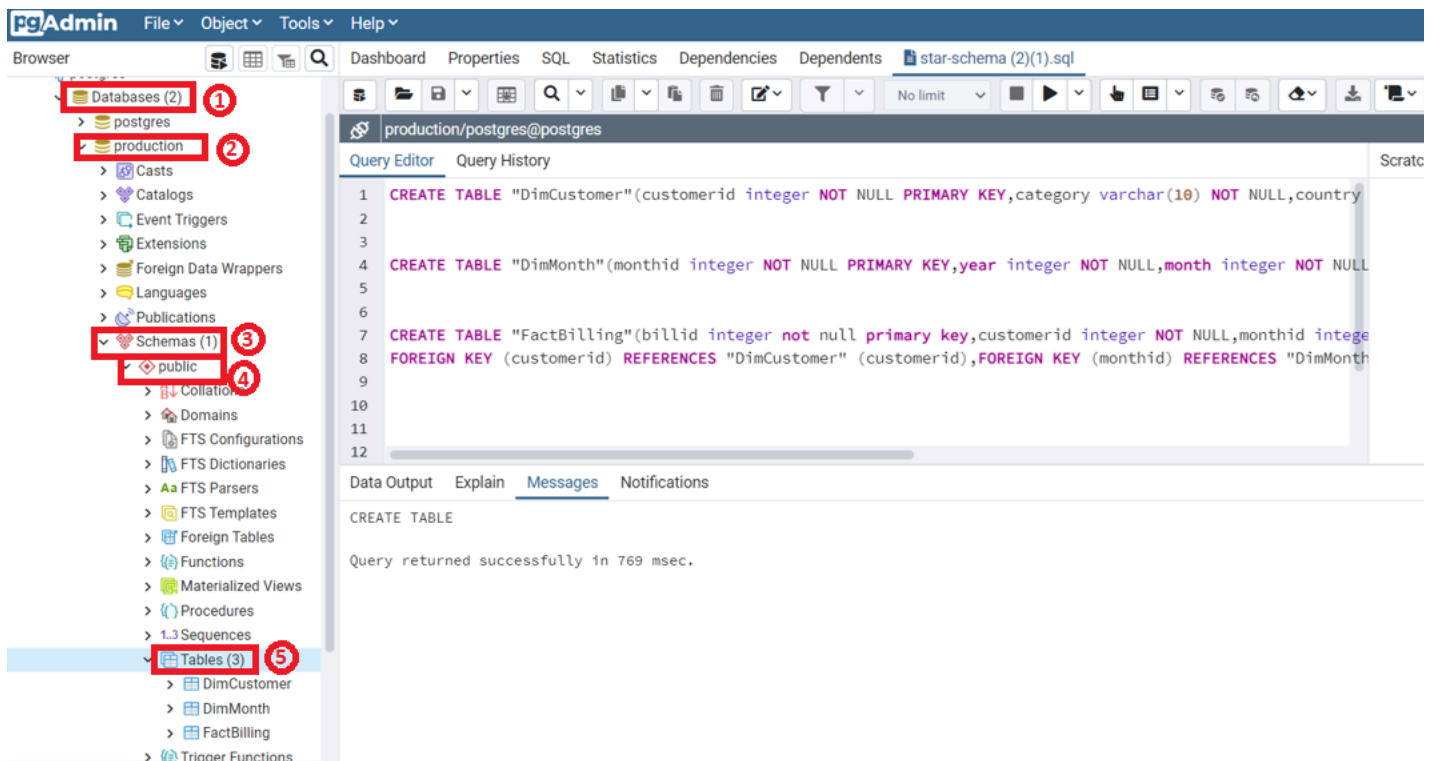
4. Once the file opens up click on the **Run** option to execute the **star-schema.sql** file.



5. Next, right-click on the **Production database** and click on the **Refresh** option from the dropdown.

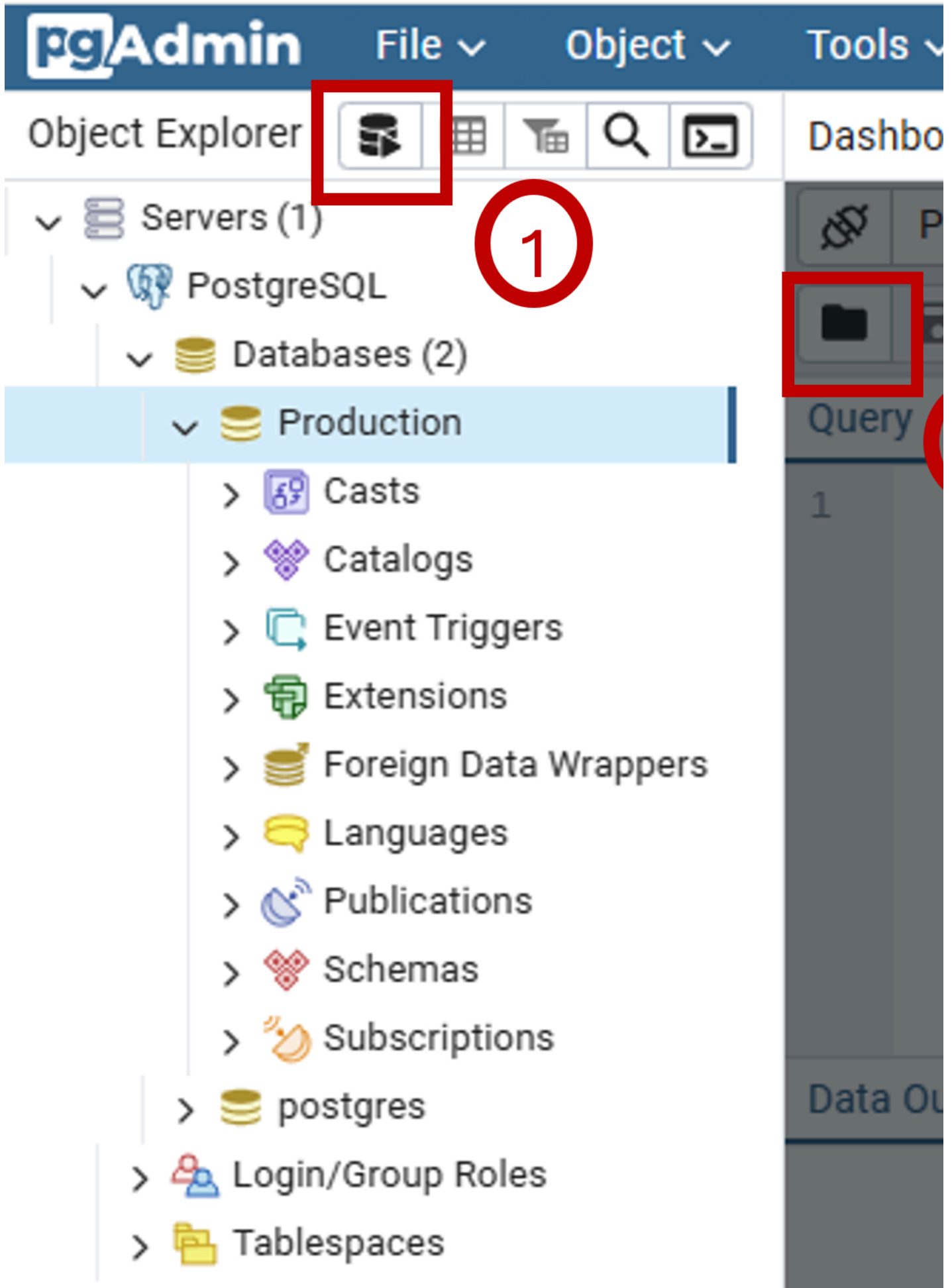


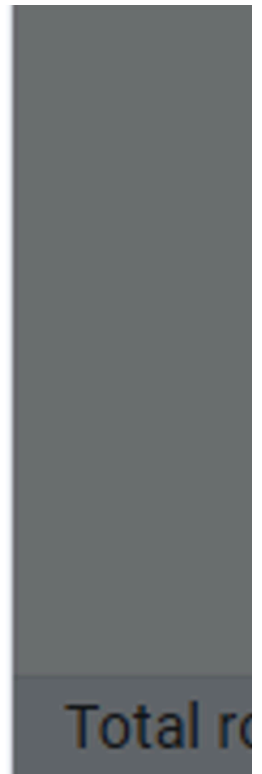
After the database is refreshed the 3 tables (DimCustomer, DimMonth, FactBilling) are created under the **Databases > Production > Schemas > Public > Tables**.



Task C: Load tables

1. Click on **Query tool** and then click **Open file** and click on the three dot and choose **Upload** option.





2. In the new blank page that appears drag and drop the **DimCustomer.sql** file inside the blank page. Once the **DimCustomer.sql** file is successfully loaded.

Click on the small X icon on the right hand side of the page as shown in the screenshot.

pgAdmin File ▾ Object ▾ Tools ▾

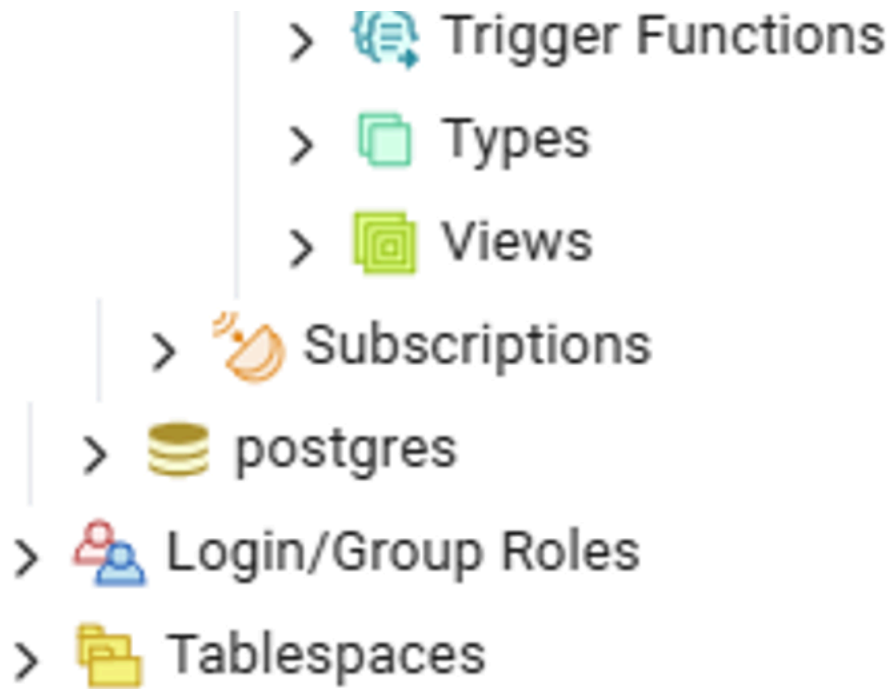
Object Explorer      × Properties

- >  Collations
- >  Domains
- >  FTS Configurations
- >  FTS Dictionaries
- >  FTS Parsers
- >  FTS Templates
- >  Foreign Tables
- >  Functions
- >  Materialized Views
- >  Operators
- >  Procedures
- >  1..3 Sequences
- ▼  **Tables (3)**
 - >  DimCustomer
 - >  DimMonth
 - >  FactBilling

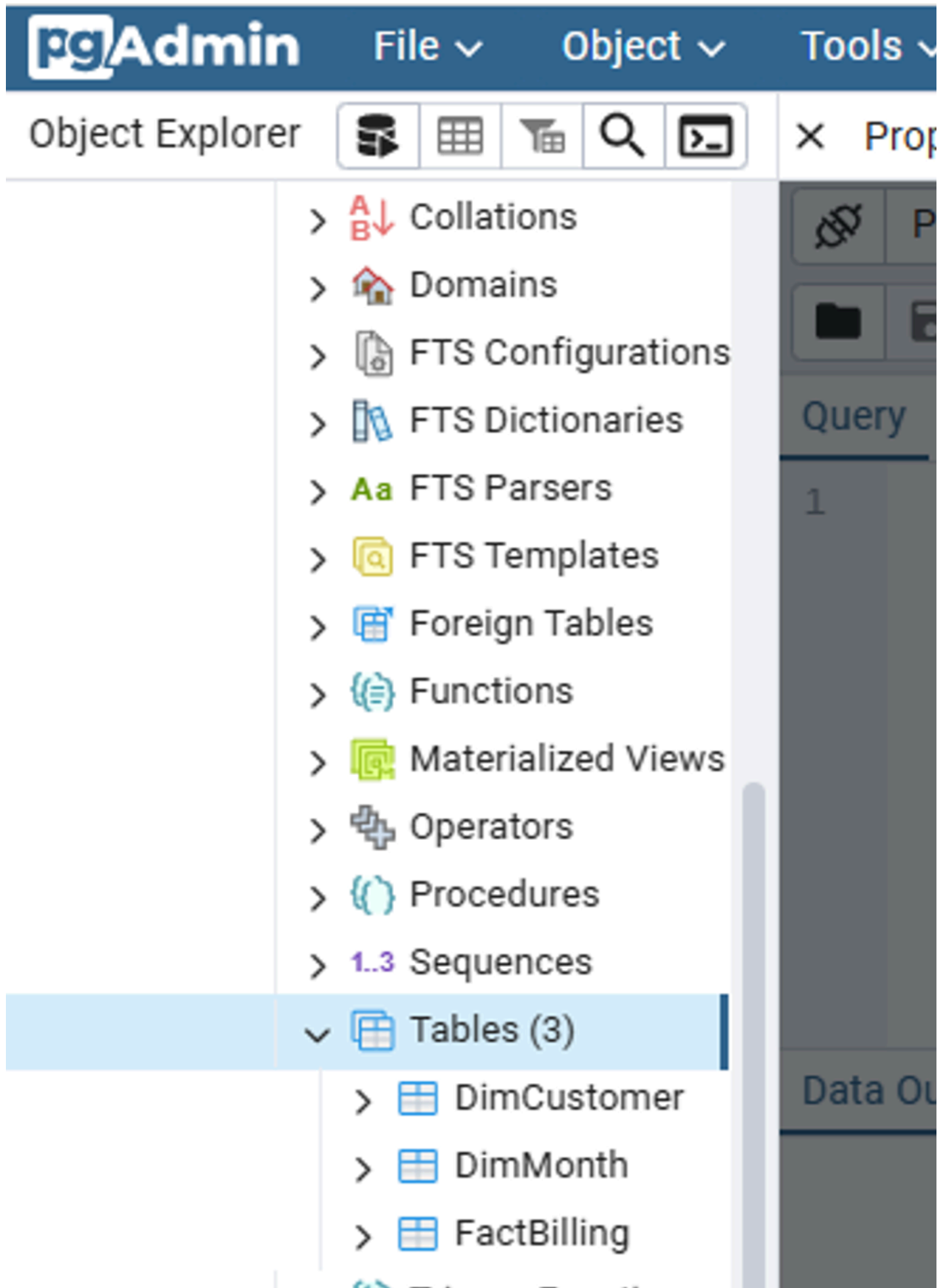
Query Editor

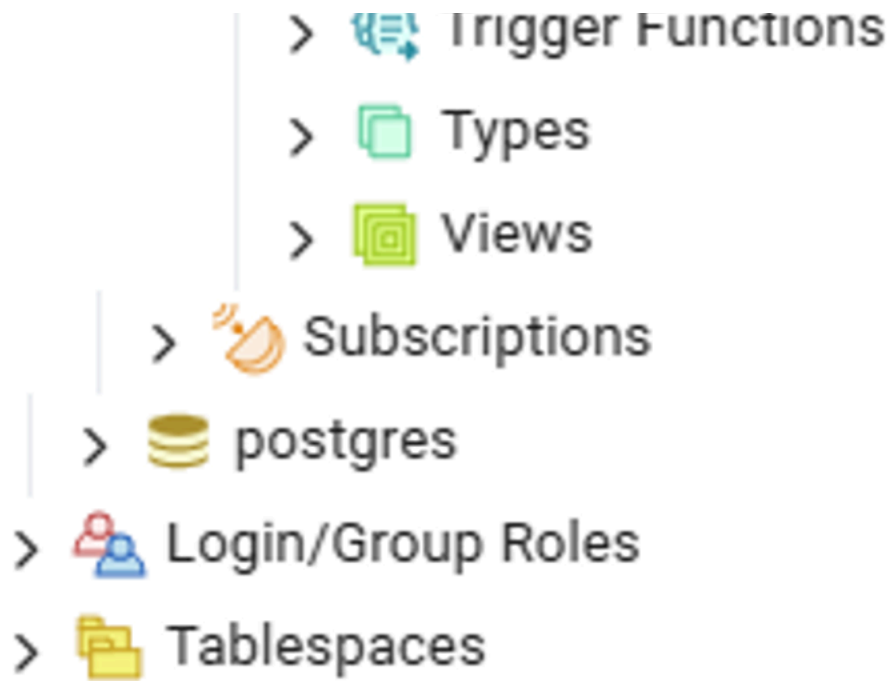
1

Data Output

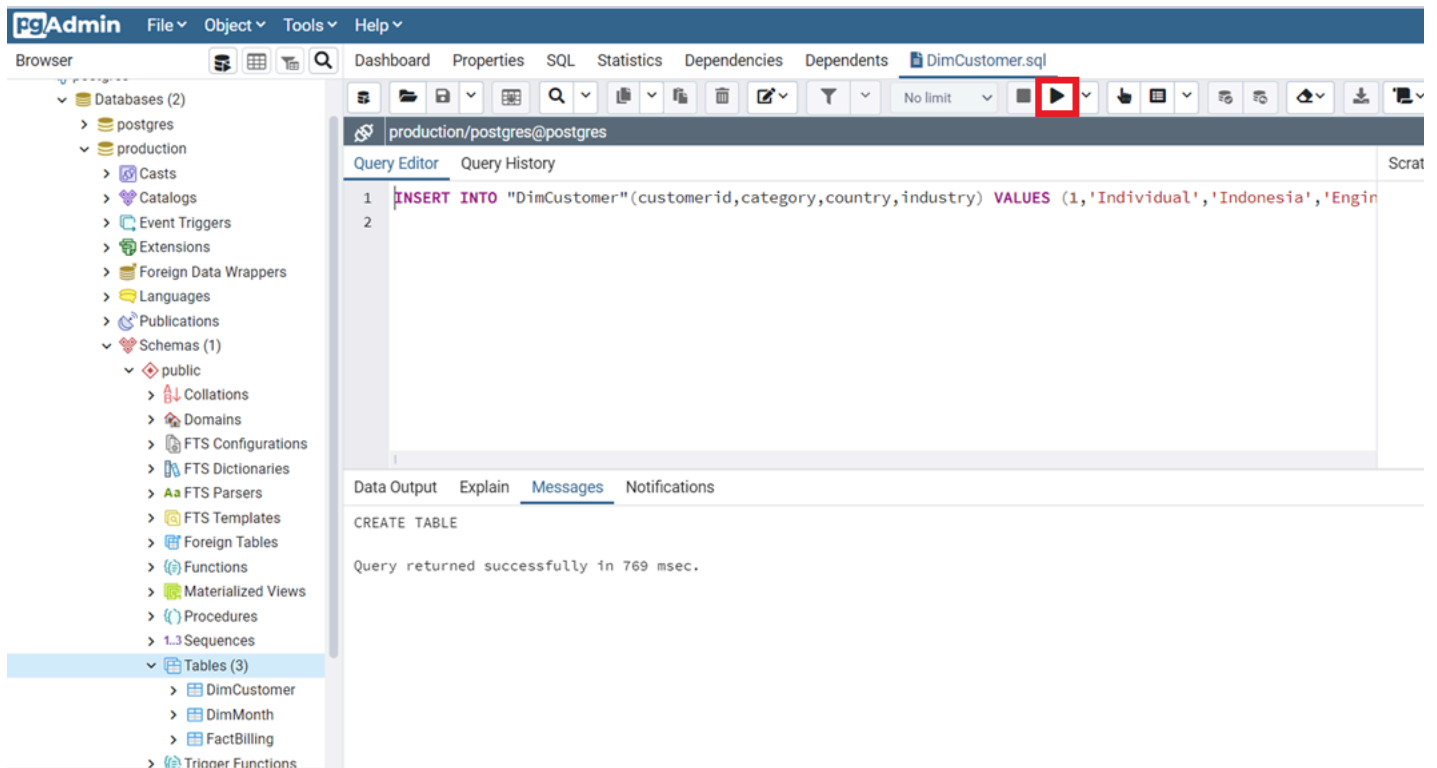


3. Once you click on the **X** icon a new page appears with the file **DimCustomer.sql**. Select the **DimCustomer.sql** file from the list and click on **Select** tab.





4. Once the file opens up, click on the **Run** option to execute the **DimCustomer.sql** file.



Note: Repeat the steps as given in Task C to upload the remaining sql files to insert data in **DimMonth** and **FactBilling**.

5. Let's run the command below on the PostgreSQL Tool.

```
select count(*) from public."DimMonth";
```

You should see an output as seen in the image below.

The screenshot shows the pgAdmin 4 web interface. On the left, the 'production' database is selected, and the 'public' schema is expanded, showing a list of tables including 'DimCustomer', 'DimMonth', and 'FactBilling'. The 'Query Editor' is active, displaying a SQL query: `select count(*) from public.\"DimMonth\";`. Below the query editor, the 'Data Output' tab shows the execution result: a single row with the column 'count' and the value '132'. A green notification at the bottom right indicates the query was successfully run.

You are encouraged to run more SQL queries.

Practice exercises

Problem 1: Using the PostgreSQL tool, find the count of rows in the table FactBilling

- ▶ [Click here for Hint](#)
- ▶ [Click here for Solution](#)

Problem 2: Using the PostgreSQL tool, create a simple Materialized views named avg_customer_bill with fields customerid and averagebillamount.

- ▶ [Click here for Hint](#)
- ▶ [Click here for Solution](#)

Problem 3: Refresh the newly created Materialized views

- ▶ [Click here for Hint](#)
- ▶ [Click here for Solution](#)

Problem 4: Using the newly created Materialized views find the customers whose average billing is more than 11000.

- ▶ [Click here for Hint](#)
- ▶ [Click here for Solution](#)

Congratulations! You have successfully finished the Populating a Data Warehouse lab.

Author

Amrutha Rao

© IBM Corporation 2022. All rights reserved.